

Subscribing to the Buzzer Topic

1. Experiment Objective

In this experiment, you will learn how to control the buzzer on the MicroROS-V2 control board through a micro-ROS topic. Through this example, you will master combining micro-ROS subscribers with hardware drivers, as well as the design pattern of using a FreeRTOS Queue to offload time-consuming operations in ROS callbacks to an independent task. This example subscribes to `UInt16` messages on the topic `/beep`, where the message value controls the buzzer ringing duration.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
micro-ROS Agent	A proxy program running on the ROS2 host side, responsible for bridging communication between the ESP32 firmware and the ROS2 system, connecting with the microcontroller via the UDP protocol
subscriber	The subscriber, the message-receiving end in ROS2 communication, subscribes to messages on a specified topic and triggers the callback function to process data when a new message arrives
rcl	A ROS2 client library for microcontrollers, providing lightweight node, publisher, subscriber and other APIs, adapted to resource-constrained embedded devices
FreeRTOS Queue	An inter-task communication mechanism provided by FreeRTOS, allowing one task to send data to a queue and another task to receive data from it, enabling decoupling and asynchronous processing
task separation pattern	A task separation design pattern that separates ROS callbacks (lightweight and fast) from hardware drivers (which may be time-consuming) into different tasks to prevent blocking the executor
QoS (Best Effort)	The "best effort" mode in the quality-of-service policy, which does not guarantee message delivery and ordering, suitable for scenarios with high real-time requirements but tolerant of a small amount of packet loss

2. Technical Architecture

This experiment builds a control chain from the ROS2 host to the ESP32 buzzer hardware:

- **Hardware layer:** The MicroROS-V2 control board (ESP32), with the buzzer connected to GPIO47 and driven by the LEDC PWM peripheral
- **Transport layer:** Uses the UDP protocol, communicating with the ROS2 host through the micro-ROS Agent
- **Node configuration:** Node name `beep_subscriber`, subscribing to topic `/beep`

- **Message type:** `std_msgs/msg/UInt16`, carrying an unsigned 16-bit integer representing the ringing duration
- **QoS mode:** Best Effort, adapting to the unreliable characteristics of the Wi-Fi UDP link
- **Data flow:** ROS2 host publishes a message → Agent → UDP → micro-ROS stack → executor → subscription callback (delivered to the FreeRTOS queue) → buzzer worker thread → `Beep_On_Time()` → PWM hardware control
- **Design pattern:** The subscription callback only delivers to the queue (microsecond level), while the hardware driver executes in an independent task, without blocking the executor

3. Core Topic Quick Reference

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
<code>/beep</code>	<code>std_msgs/msg/UInt16</code>	Best Effort	<code>data</code> (uint16)	0=off, 1=continuous ringing, ≥10=ring for xx ms	Input, controls the buzzer

Usage Example:

```
# The buzzer rings for 1000 ms
ros2 topic pub /beep std_msgs/msg/UInt16 "data: 1000"
# Turn off the buzzer
ros2 topic pub /beep std_msgs/msg/UInt16 "data: 0"
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

```
| RMW_UXRCE_TRANSPORT | udp | UDP transport (Wi-Fi communication) |
| RMW_UXRCE_MAX_PUBLISHERS | 6 | Maximum number of publishers |
| RMW_UXRCE_MAX_SUBSCRIPTIONS | 6 | Maximum number of subscribers |
| RMW_UXRCE_MAX_NODES | 1 | Maximum number of nodes |
| RMW_UXRCE_MAX_SERVICES | 0 | Services not supported |
| RMW_UXRCE_MAX_CLIENTS | 0 | Clients not supported |
| RMW_UXRCE_MAX_HISTORY | 1 | History depth = 1, no message queuing |
| UCLIENT_UDP_TRANSPORT_MTU | 4096 | Maximum UDP packet size 4 KB |
| UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL | 1 | Agent heartbeat interval 1 ms |
```

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0, MAX_CLIENTS=0` means only topic communication is supported, not Service/Client

- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

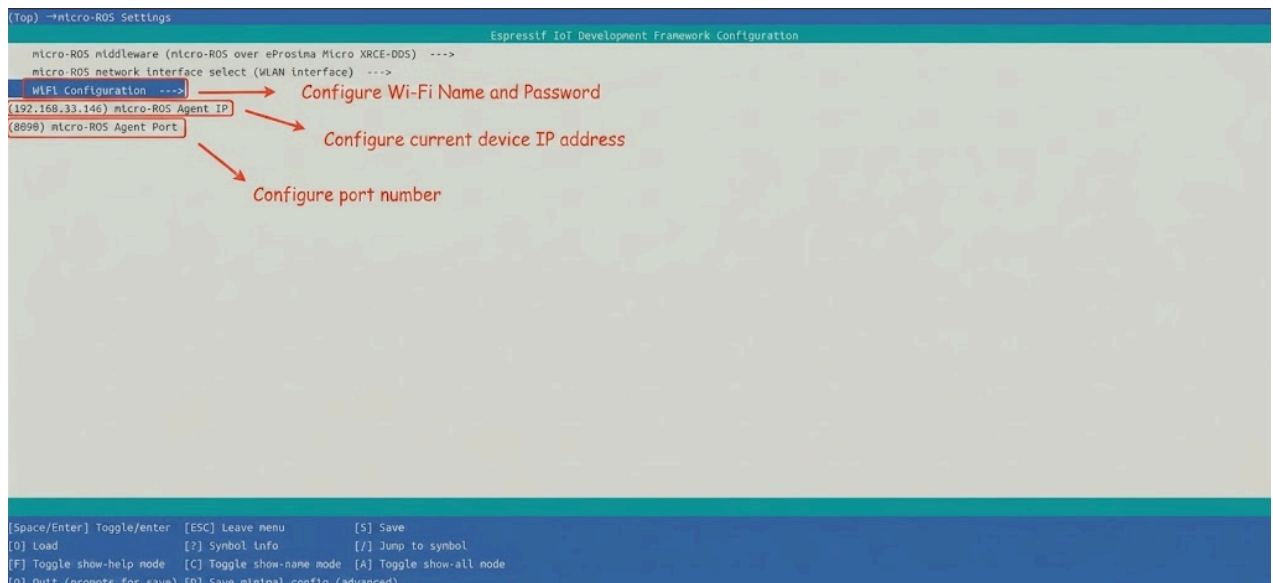
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB Type-C cable	✓ Required
Wi-Fi network (2.4 GHz)	✓ Required

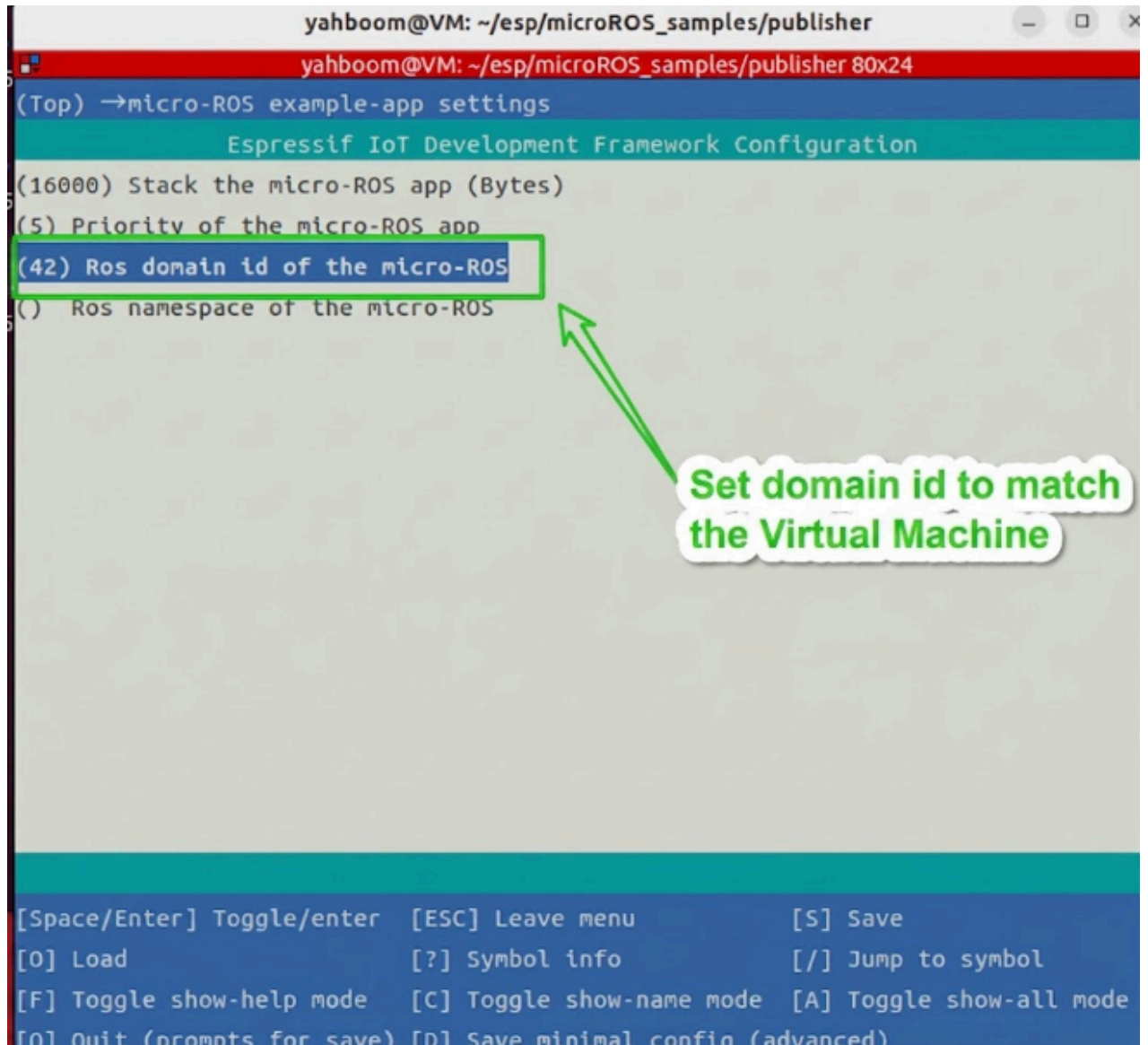
2. Prerequisites

- The ESP32-IDF development environment has been set up
- The MicroROS-V2 control board is connected to the computer via USB
- Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/beep_subscriber
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



- Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/beep_subscriber
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

- 1. Press `Ctrl+]` in Terminal 2 to exit the serial monitor (stop firmware execution)
- 2. Press `Ctrl+C` in Terminal 1 to stop `micro_ros_agent`

4. Experiment Result

After the build and flash succeed, publish buzzer commands on the ROS2 host:

```
# The buzzer rings for 1000 ms
ros2 topic pub --once /beep std_msgs/msg/UInt16 "data: 1000"

# The buzzer rings continuously
ros2 topic pub --once /beep std_msgs/msg/UInt16 "data: 1"

# Turn off the buzzer
ros2 topic pub --once /beep std_msgs/msg/UInt16 "data: 0"
```

Meaning of message values:

data value	Behavior
0	Turn off the buzzer
1	The buzzer rings continuously
≥10	Ring for the specified number of milliseconds, then turn off automatically (limited to the range 10~10000)

The control board serial port will output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: free heap before support init: xxx
I (xxx) MAIN: stack watermark before support init: xxx
I (xxx) MAIN: free heap after support init: xxx
I (xxx) MAIN: stack watermark after support init: xxx
I (xxx) MAIN: micro-ROS beep subscriber init done
Beep command: 1000
I (xxx) BEEP: Start Beep_Task with core:1
```

5. Program Analysis

Source code paths:

- Firmware source code: `~/esp/microROS_samples/beep_subscriber/main/main.c`
- Buzzer driver: `~/esp/microROS_samples/beep_subscriber/components/beep/beep.h`, `beep.c`

1. Main Program Logic

`main/main.c` is the program entry point. Unlike the basic subscriber, this example adopts a **task separation design pattern**, decoupling ROS communication from hardware control:

1. In `app_main()`:
- Calls `Beep_Init()` to initialize the buzzer GPIO and PWM peripherals

◦ Creates the FreeRTOS queue `s_beep_queue` (length 1)

◦ Creates the `beep_worker_task` task, which is responsible for the actual buzzer driving

◦ Initializes the Wi-Fi network

◦ Creates the `micro_ros_task` task, which is responsible for micro-ROS communication
2. In `micro_ros_task`:
- Initializes micro-ROS and creates the node `beep_subscriber`

◦ Subscribes to the topic `beep` in Best Effort mode, with message type `std_msgs/msg/UInt16`

◦ The subscription callback `beep_callback` only forwards the message through the queue, without performing time-consuming operations
3. In `beep_worker_task`:
- Receives buzzer commands from the queue

◦ Calls `Beep_On_Time()` to execute the actual hardware control

2. Key Components

`components/beep/beep.h` — Buzzer driver interface:

Parameter definition file: `~/esp/microROS_samples/beep_subscriber/components/beep/beep.h`

Pin/Parameter	Value	Description
<code>BEEP_GPIO</code>	47	GPIO pin the buzzer is connected to
<code>BEEP_DEFAULT_FREQ_HZ</code>	2000	Default PWM frequency (passive buzzer)
<code>BEEP_DEFAULT_DUTY_PCT</code>	50	Default duty cycle 50%

`components/beep/beep.c` — Core buzzer driver functions:

Function	Description
<code>Beep_Init()</code>	Initializes the LEDC PWM peripheral (Timer 0, Channel 0, 10-bit resolution) and creates the background <code>Beep_Task</code>
<code>Beep_On()</code>	Rings the buzzer continuously
<code>Beep_Off()</code>	Turns off the buzzer
<code>Beep_On_Time(uint16_t time)</code>	Sets the ringing duration: 0=off, 1=continuous ringing, ≥10=ring for xx ms then turn off automatically

Function	Description
<code>Beep_Handle()</code>	Periodic processing function, called every 10 ms by the background task, managing the delayed turn-off logic
<code>Beep_Set_Freq(uint16_t freq_hz)</code>	Sets the PWM frequency (100~10000 Hz)
<code>Beep_Get_Freq()</code>	Gets the current PWM frequency

Design of separating the subscription callback from the executor:

Source file: `~/esp/microROS_samples/beep_subscriber/main/main.c`

```
/* Subscription callback: only receives the message and delivers the command, without time-
consuming operations */
static void beep_callback(const void *msgin)
{
    const std_msgs__msg__UInt16 *msg = (const std_msgs__msg__UInt16 *)msgin;
    uint16_t beep_time = msg->data;

    if (s_beep_queue != NULL) {
        /* Queue length is 1, always keeps the latest command */
        (void)xQueueOverwrite(s_beep_queue, &beep_time);
    }
}

/* Buzzer execution task: moves time-consuming actions out of the ROS callback */
static void beep_worker_task(void *arg)
{
    (void)arg;
    uint16_t beep_time = 0;
    while (1) {
        if (xQueueReceive(s_beep_queue, &beep_time, portMAX_DELAY) == pdTRUE) {
            printf("Beep command: %u\n", (unsigned)beep_time);
            Beep_On_Time(beep_time);
        }
    }
}
```

Benefits of the task separation design:

- Time-consuming operations cannot be executed in ROS callbacks (they would block the executor)
- `xQueueOverwrite` ensures the queue always holds the latest command (old commands are overwritten)
- The buzzer execution task can safely call the driver functions

Topic configuration:

Parameter definition file: `~/esp/microROS_samples/beep_subscriber/main/main.c`

Configuration Item	Value	Description
Node name	<code>beep_subscriber</code>	ROS2 node name
Topic name	<code>beep</code>	Name of the subscribed topic
Message type	<code>std_msgs/msg/UInt16</code>	Unsigned 16-bit integer, representing the ringing duration (ms)
QoS mode	Best Effort	Reduces pressure on the Wi-Fi UDP link

6. Common Problems

Problem	Cause	Solution
The buzzer does not ring	PWM frequency or duty cycle configuration problem	Default frequency 2000 Hz, duty cycle 50%; check whether the hardware supports a passive buzzer
No buzzer response after publishing a message	Topic name mismatch or the Agent is not connected	Confirm the topic name is <code>/beep</code> and the message type is <code>std_msgs/msg/UInt16</code>
Abnormal buzzer sound	Inappropriate PWM frequency	Use <code>Beep_Set_Freq()</code> to adjust the frequency (100~10000 Hz)
Sending multiple commands consecutively only executes the last one	Design behavior of a queue with length 1	This is expected behavior; the latest command is always executed